



BHARATIYA VIDYA BHAVAN'S

SARDAR PATEL INSTITUTE OF TECHNOLOGY

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai – 400058-India

Department of Computer Engineering

Name	Siddh Sakariya
UID no.	2024300200
Experiment No.	8

AIM:	To study and configure firewall rules in Linux using iptables and understand packet filtering mechanisms.
THEORY:	iptables is a command-line utility used to configure and manage the Linux kernel firewall (netfilter). It allows users to define rules that control incoming, outgoing, and forwarded network traffic. iptables works using tables, chains, and rules: • Tables: filter (default), nat, mangle • Chains: ◦ INPUT → incoming traffic ◦ OUTPUT → outgoing traffic ◦ FORWARD → routed traffic • Targets: ◦ ACCEPT → allow packet ◦ DROP → block packet ◦ REJECT → block with response iptables enables filtering based on: • IP address • Protocol (TCP, UDP, ICMP) • Port number This helps in implementing security policies and controlling network access in distributed systems.



Program 1	
PROBLEM STATEMENT :	This exercise will teach you some iptables essentials. In particular, you will learn how to view or list iptables rules, create basic filtering rules, delete rules, create/delete custom chains and so on. In this exercise, you are required to explore the core functionalities of the <code>iptables</code> tool used for configuring firewall rules in a Linux environment. The task involves examining existing firewall configurations, creating and modifying filtering rules, and managing rule chains. Specifically, you will: ● Inspect and interpret existing iptables rules across different tables and chains ● Create and apply basic packet filtering rules ● Delete and modify existing firewall rules ● Create and manage user-defined chains ● Flush existing rules to reset configurations



OUTPUT:

1 View all rules

iptables -L

```
students@students-HP-280-G3-M1:~/jps$ sudo -i
[sudo] password for students:
root@students-HP-280-G3-MT:~# iptables -L
Chain INPUT (policy DROP)
target     prot opt source                destination
ACCEPT    tcp  --  anywhere               anywhere             tcp dpt:domain /* generated for Multipass network mpqemubr0 */
ACCEPT    udp  --  anywhere               anywhere             udp dpt:domain /* generated for Multipass network mpqemubr0 */
ACCEPT    udp  --  anywhere               anywhere             udp dpt:bootps /* generated for Multipass network mpqemubr0 */
LIBVIRT_INP all  --  anywhere               anywhere
ufw-before-logging-input all -- anywhere               anywhere
ufw-before-input all -- anywhere               anywhere
ufw-after-input all -- anywhere               anywhere
ufw-after-logging-input all -- anywhere               anywhere
ufw-reject-input all -- anywhere               anywhere
ufw-track-input all -- anywhere               anywhere

Chain FORWARD (policy DROP)
target     prot opt source                destination
ACCEPT    all  --  anywhere               anywhere             /* generated for Multipass network mpqemubr0 */
ACCEPT    all  --  10.232.115.0/24         anywhere             /* generated for Multipass network mpqemubr0 */
ACCEPT    all  --  anywhere               10.232.115.0/24      ctstate RELATED,ESTABLISHED /* generated for Multipass network mpqemubr0 */
DOCKER-USER all -- anywhere               anywhere
DOCKER-FORWARD all -- anywhere               anywhere
LIBVIRT_FWX all -- anywhere               anywhere
LIBVIRT_FWI all -- anywhere               anywhere
LIBVIRT_FWO all -- anywhere               anywhere
```

2. Verbose

output iptables

-L -v

```
Chain ufw-user-output (1 references)
target     prot opt source                destination
ACCEPT    all  --  anywhere               anywhere
Chain INPUT (policy DROP 612 packets, 19632 bytes)
pkts bytes target     prot opt in     out     source                destination
0      0 ACCEPT    tcp  --  mpqemubr0 any    anywhere             tcp dpt:domain /* generated for Multipass network mpqemubr0 */
18    698 ACCEPT    udp  --  mpqemubr0 any    anywhere             udp dpt:domain /* generated for Multipass network mpqemubr0 */
0      0 ACCEPT    udp  --  mpqemubr0 any    anywhere             udp dpt:bootps /* generated for Multipass network mpqemubr0 */
118K  107M LIBVIRT_INP all  --  any    any    anywhere             anywhere
118K  107M ufw-before-logging-input all -- any    any    anywhere             anywhere
118K  107M ufw-before-input all -- any    any    anywhere             anywhere
252K  388K ufw-after-input all -- any    any    anywhere             anywhere
631  20240 ufw-after-logging-input all -- any    any    anywhere             anywhere
631  20240 ufw-reject-input all -- any    any    anywhere             anywhere
631  20240 ufw-track-input all -- any    any    anywhere             anywhere

Chain FORWARD (policy DROP 0 packets, 0 bytes)
pkts bytes target     prot opt in     out     source                destination
2    104 ACCEPT    all  --  mpqemubr0 mpqemubr0 anywhere             anywhere             /* generated for Multipass network mpqemubr0 */
135K 74940 ACCEPT    all  --  mpqemubr0 any    10.232.115.0/24      anywhere             /* generated for Multipass network mpqemubr0 */
2020 3007K ACCEPT    all  --  any    mpqemubr0 anywhere             10.232.115.0/24      ctstate RELATED,ESTABLISHED /* generated for Multipass network mpqemubr0 */
10839 9543K DOCKER-USER all -- any    any    anywhere             anywhere
10839 9543K DOCKER-FORWARD all -- any    any    anywhere             anywhere
0      0 LIBVIRT_FWX all -- any    any    anywhere             anywhere
0      0 LIBVIRT_FWI all -- any    any    anywhere             anywhere
0      0 LIBVIRT_FWO all -- any    any    anywhere             anywhere
0      0 ufw-before-logging-forward all -- any    any    anywhere             anywhere
0      0 ufw-before-forward all -- any    any    anywhere             anywhere
0      0 ufw-after-forward all -- any    any    anywhere             anywhere
0      0 ufw-after-logging-forward all -- any    any    anywhere             anywhere
0      0 ufw-reject-forward all -- any    any    anywhere             anywhere
0      0 ufw-track-forward all -- any    any    anywhere             anywhere
0      0 REJECT    all  --  mpqemubr0 any    anywhere             anywhere             /* generated for Multipass network mpqemubr0 */ re
ject-with icmp-port-unreachable
```

3. VIEW MANGLE



```
root@students-HP-280-G3-MT:~# iptables -L -t mangle
Chain PREROUTING (policy ACCEPT)
target prot opt source destination

Chain INPUT (policy ACCEPT)
target prot opt source destination

Chain FORWARD (policy ACCEPT)
target prot opt source destination

Chain OUTPUT (policy ACCEPT)
target prot opt source destination

Chain POSTROUTING (policy ACCEPT)
target prot opt source destination
CHECKSUM udp -- anywhere anywhere udp dpt:bootpc /* generated for Multipass network mpqemu0 */ CHECKSUM flll
LIBVIRT_PRT all -- anywhere anywhere

Chain LIBVIRT_PRT (1 references)
target prot opt source destination
CHECKSUM udp -- anywhere anywhere udp dpt:bootpc CHECKSUM flll
root@students-HP-280-G3-MT:~#
```

4. View NAT

table iptables -L -t

```
root@students-HP-280-G3-MT:~# iptables -L -t nat
Chain PREROUTING (policy ACCEPT)
target prot opt source destination
DOCKER all -- anywhere anywhere ADDTYPE match dst-type LOCAL

Chain INPUT (policy ACCEPT)
target prot opt source destination

Chain OUTPUT (policy ACCEPT)
target prot opt source destination
DOCKER all -- anywhere !localhost/8 ADDTYPE match dst-type LOCAL

Chain POSTROUTING (policy ACCEPT)
target prot opt source destination
MASQUERADE all -- 10.232.115.0/24 110.232.115.0/24 /* generated for Multipass network mpqemu0 */
MASQUERADE udp -- 10.232.115.0/24 110.232.115.0/24 /* generated for Multipass network mpqemu0 */ nsq ports: 3024-65535
MASQUERADE tcp -- 10.232.115.0/24 110.232.115.0/24 /* generated for Multipass network mpqemu0 */ nsq ports: 3024-65535
RETURN all -- 10.232.115.0/24 255.255.255.255 /* generated for Multipass network mpqemu0 */
RETURN all -- 10.232.115.0/24 base-address.mcast.net/24 /* generated for Multipass network mpqemu0 */
MASQUERADE all -- 172.19.0.0/16 anywhere
MASQUERADE all -- 192.168.49.0/24 anywhere
MASQUERADE all -- 172.19.0.0/16 anywhere
MASQUERADE all -- 172.21.0.0/16 anywhere
MASQUERADE all -- 172.28.0.0/16 anywhere
MASQUERADE all -- 172.18.0.0/16 anywhere
LIBVIRT_PRT all -- anywhere anywhere

Chain DOCKER (2 references)
target prot opt source destination
RETURN all -- anywhere anywhere
RETURN all -- anywhere anywhere
RETURN all -- anywhere anywhere
RETURN all -- anywhere anywhere
RETURN all -- anywhere anywhere
```

5. Flush all

rules iptables -F

```
root@students-HP-280-G3-MT:~# iptables -F
```

6. Create custom

chain iptables -N

mychain

```
Chain mychain (0 references)
target prot opt source destination
```



Listing current iptable chains

```
root@students-HP-280-G3-MT:~# iptables -L INPUT -v
chain INPUT (policy DROP 625 packets, 28048 bytes)
pkts bytes target prot opt in out source destination
0 0 ACCEPT tcp -- npqenubr0 any anywhere anywhere tcp dpt:domain /* generated for Multipass network
npqenubr0 */
10 698 ACCEPT udp -- npqenubr0 any anywhere anywhere udp dpt:domain /* generated for Multipass network
npqenubr0 */
0 0 ACCEPT udp -- npqenubr0 any anywhere anywhere udp dpt:bootps /* generated for Multipass network
npqenubr0 */
129K 108M LIBVIRT_INP all -- any any anywhere anywhere anywhere
126K 108M ufw-before-logging-input all -- any any anywhere anywhere anywhere
126K 108M ufw-before-input all -- any any anywhere anywhere anywhere
2672 414K ufw-after-input all -- any any anywhere anywhere anywhere
644 20656 ufw-after-logging-input all -- any any anywhere anywhere anywhere
644 20656 ufw-reject-input all -- any any anywhere anywhere anywhere
644 20656 ufw-track-input all -- any any anywhere anywhere anywhere
root@students-HP-280-G3-MT:~#
```

10. Delete built-in chain

```
root@students-HP-280-G3-MT:~# iptables -X INPUT
iptables: Invalid argument. Run 'dmesg' for more information.
root@students-HP-280-G3-MT:~# sudo iptables -X INPUT
iptables: Invalid argument. Run 'dmesg' for more information.
root@students-HP-280-G3-MT:~#
```

11. Deleting of custom chain and

```
root@students-HP-280-G3-MT:~# iptables -X mychain
root@students-HP-280-G3-MT:~# iptables -L mychain
iptables: No chain/target/match by that name.
root@students-HP-280-G3-MT:~# sudo iptables -L mychain
iptables: No chain/target/match by that name.
root@students-HP-280-G3-MT:~#
```

RESULT:

The NAT table is mainly used for modifying IP addresses of packets, especially during operations like Network Address Translation. It includes chains such as PREROUTING, POSTROUTING, and OUTPUT, which determine at what stage the changes are applied.

The mangle table is used for performing advanced modifications on packets, such as altering packet headers for specialized networking requirements.

A **chain** is essentially a sequence of rules that packets pass through. The commonly used chains are:

- **INPUT** for incoming packets
- **OUTPUT** for outgoing packets
- **FORWARD** for packets that are routed through the system



	Each rule in a chain consists of certain matching conditions, such as protocol type, IP address, or port number, along with an associated
--	---

	action (target). The most commonly used targets are: • ACCEPT, which allows the packet • DROP, which blocks the packet silently • REJECT, which blocks the packet and sends a response This exercise helps in building a basic understanding of firewall configuration by showing how packets are processed step by step through different tables and chains. It also highlights how adding or modifying rules directly impacts the flow of network traffic in real time
--	--

Program 2	
PROBLEM STATEMENT :	Basic Packet Filtering This exercise will teach you how to craft slightly more advanced packet filtering rules. In particular, you will block all ICMP packet types from your partner system.
PROGRAM:	PING dropped <pre>root@students-HP-280-G3-MT:~# iptables -A INPUT -p icmp -s 192.168.1.3 -j DROP root@students-HP-280-G3-MT:~# ping 192.168.1.3 PING 192.168.1.3 (192.168.1.3) 56(84) bytes of data. ^Z [1]+ Stopped ping 192.168.1.3 root@students-HP-280-G3-MT:~#</pre> Iptables flushed and dropped <pre>root@students-HP-280-G3-MT:~# iptables -F root@students-HP-280-G3-MT:~# iptables -A OUTPUT -p icmp -j DROP root@students-HP-280-G3-MT:~# ping -c 2 8.8.8.8 PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data. ping: sendmsg: Operation not permitted ping: sendmsg: Operation not permitted --- 8.8.8.8 ping statistics --- 2 packets transmitted, 0 received, 100% packet loss, time 1004ms root@students-HP-280-G3-MT:~#</pre>



BHARATIYA VIDYA BHAVAN'S

SARDAR PATEL INSTITUTE OF TECHNOLOGY

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai – 400058-India

Department of Computer Engineering

```
root@students-HP-280-G3-MT:~# ping -c 2 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=2.04 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=1.87 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 1.866/1.952/2.039/0.086 ms
root@students-HP-280-G3-MT:~#
```

```
root@students-HP-280-G3-MT:~# iptables -A INPUT -p icmp -s 8.8.8.8 -j DROP
root@students-HP-280-G3-MT:~# iptables -L INPUT -v
Chain INPUT (policy ACCEPT 38 packets, 4425 bytes)
pkts bytes target prot opt in out source destination
0 0 DROP icmp -- any any dns.google anywhere
root@students-HP-280-G3-MT:~#
```

Ip tables rules different changed

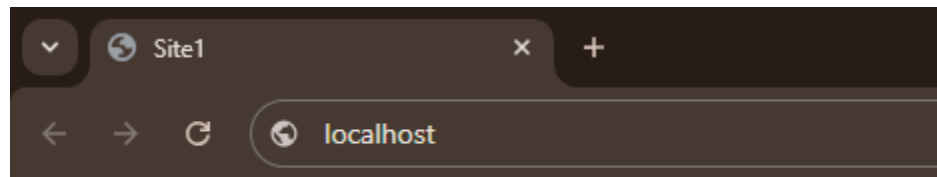
```
root@students-HP-280-G3-MT:~# iptables -L OUTPUT --line-numbers
Chain OUTPUT (policy ACCEPT)
num target prot opt source destination
1 DROP icmp -- anywhere anywhere
root@students-HP-280-G3-MT:~# iptables -D OUTPUT 1
root@students-HP-280-G3-MT:~# ping -c 2 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=116 time=1.35 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=116 time=1.22 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 1.222/1.285/1.348/0.063 ms
root@students-HP-280-G3-MT:~#
```

```
[[1]] stopped ping 192.168.1.3
root@students-HP-280-G3-MT:~# iptables -L INPUT -v
Chain INPUT (policy ACCEPT 6170 packets, 2028K bytes)
pkts bytes target prot opt in out source destination
0 0 DROP icmp -- any any dns.google anywhere
155 13020 DROP icmp -- any any students-HP-280-G3-MT anywhere
root@students-HP-280-G3-MT:~#
```

HTTP:

HTTP working fine



This is Site1



RESULT:	After modifying the firewall rules, the ping command fails, indicating that ICMP traffic is successfully blocked as per the configured rule. This experiment demonstrates how network traffic can be controlled using different filtering techniques. Packet filtering works by examining parameters such as source and destination IP addresses, the type of protocol (ICMP, TCP, UDP), and port numbers before deciding whether to allow or block the packet. ICMP is mainly used for operations like ping and basic network diagnostics. Blocking ICMP traffic prevents systems from responding to ping requests, which can improve security by making the system less visible on the network. TCP-based filtering is used to control access to common services. For example, FTP operates on port 21 and HTTP on port 80. By applying rules on these ports, specific services can be restricted or completely blocked. Traffic control can be applied in two ways: • The INPUT chain is used to filter incoming traffic • The OUTPUT chain is used to control outgoing traffic Filtering can also be done based on specific IP addresses, allowing the system to block or allow traffic from selected hosts. Similarly, port-based filtering helps in managing access to individual services. Another important observation is that iptables processes rules in a sequential manner, from top to bottom. This means that the order of rules is important, and incorrect placement can affect the final outcome. Individual rules can also be modified or removed using their line numbers.
----------------	---

Program 3	
PROBLEM STATEMENT :	Basic Packet Forwarding In this exercise you will learn how to set up a basic packet forwarding rule. The rule you set up will allow your system to serve as a router to your partner system. You system will route all traffic that originates from your partner's system to the internet or to your own default gateway. This is what is known as IP masquerading or NAT (Network address translation). To be pedantic, IP masquerading and NAT-ing are actually slightly different beasts and are usually used for accomplishing different things. We won't dwell too much on the specific differences in the following exercises



BHARATIYA VIDYA BHAVAN'S

SARDAR PATEL INSTITUTE OF TECHNOLOGY

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai – 400058-India

Department of Computer Engineering

	In this experiment, ServerXY acts as a router between: • External Network → 172.16.0.0/16 (via eth0) • Internal Network → 10.0.0.0/8 (via eth1) ServerPR is a client in the internal network. Since private IPs (10.0.0.0/8) cannot access the internet directly: • NAT (MASQUERADE) is used to translate private IP → external IP • IP forwarding allows packets to pass through ServerXY
PROGRAM:	



```
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.  
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=1.04 ms  
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=1.57 ms  
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=1.29 ms  
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=1.67 ms  
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=2.41 ms  
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=1.84 ms  
64 bytes from 10.0.0.1: icmp_seq=7 ttl=64 time=3.16 ms  
64 bytes from 10.0.0.1: icmp_seq=8 ttl=64 time=2.30 ms  
64 bytes from 10.0.0.1: icmp_seq=9 ttl=64 time=1.58 ms  
64 bytes from 10.0.0.1: icmp_seq=10 ttl=64 time=1.78 ms  
64 bytes from 10.0.0.1: icmp_seq=11 ttl=64 time=1.76 ms  
64 bytes from 10.0.0.1: icmp_seq=12 ttl=64 time=1.46 ms  
64 bytes from 10.0.0.1: icmp_seq=13 ttl=64 time=1.96 ms  
64 bytes from 10.0.0.1: icmp_seq=14 ttl=64 time=1.94 ms  
64 bytes from 10.0.0.1: icmp_seq=15 ttl=64 time=1.70 ms  
64 bytes from 10.0.0.1: icmp_seq=16 ttl=64 time=1.58 ms  
64 bytes from 10.0.0.1: icmp_seq=17 ttl=64 time=1.02 ms  
64 bytes from 10.0.0.1: icmp_seq=18 ttl=64 time=1.12 ms  
64 bytes from 10.0.0.1: icmp_seq=19 ttl=64 time=1.74 ms  
^C  
--- 10.0.0.1 ping statistics ---  
19 packets transmitted, 19 received, 0% packet loss, time 20238ms  
rtt min/avg/max/mdev = 1.017/1.732/3.158/0.493 ms
```

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.  
From 10.0.0.1: icmp_seq=2 Redirect Host(New nexthop: 8.8.8.8)  
From 10.0.0.1: icmp_seq=3 Redirect Host(New nexthop: 8.8.8.8)  
From 10.0.0.1: icmp_seq=4 Redirect Host(New nexthop: 8.8.8.8)  
From 10.0.0.1: icmp_seq=5 Redirect Host(New nexthop: 8.8.8.8)  
From 10.0.0.1: icmp_seq=6 Redirect Host(New nexthop: 8.8.8.8)  
From 10.0.0.1 icmp_seq=4 Destination Host Unreachable  
From 10.0.0.1: icmp_seq=9 Redirect Host(New nexthop: 8.8.8.8)  
From 10.0.0.1 icmp_seq=8 Destination Host Unreachable  
From 10.0.0.1 icmp_seq=9 Destination Host Unreachable  
^C  
--- 8.8.8.8 ping statistics ---  
11 packets transmitted, 0 received, +3 errors, 100% packet loss, time 10930ms  
pipe 4
```

1. Verbose option for iptables `-L -t`
`nat iptables -L -t nat -v`
2. Command to display rules under OUTPUT chain



	<pre>iptables -L OUTPUT</pre> 3. Default port for FTP 21 4. Command to create chain “mynat-chain” in NAT table <pre>iptables -t nat -N mynat-chain</pre> 5. Easier firewall management tools • ufw • firewalld • nftables • GFW • iptables-persistent 6. Block HTTP traffic from hq.example.org • HTTP port: 80 • Command: <pre>iptables -A INPUT -p tcp -s 172.16.0.100 --dport 80 -j DROP</pre> • Plain English: This rule blocks all incoming TCP traffic from 172.16.0.100 trying to access port 80 (HTTP) on the local machine.
--	---



BHARATIYA VIDYA BHAVAN'S

SARDAR PATEL INSTITUTE OF TECHNOLOGY

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai – 400058-India

Department of Computer Engineering

CONCLUSION :

In this experiment, I learned how to configure and manage a firewall in Linux using `iptables`. I explored the basic concepts of tables, chains, and rules, and understood how packet filtering works in the Linux kernel through the netfilter subsystem.

In Exercise 1, I learned how to list, create, and delete rules and chains, which helped me understand how firewall rules are structured and managed.

In Exercise 2, I implemented packet filtering by blocking ICMP and TCP traffic based on protocol, IP address, and port number. This showed me how specific services like ping and FTP can be controlled using firewall rules.

In Exercise 3, I configured packet forwarding and NAT (MASQUERADE), which allowed my system to act as a router. I observed that both NAT and IP forwarding are required for successful communication between internal and external networks

Overall, this experiment gave me practical understanding of how firewall rules can be dynamically applied to control network traffic, enhance security, and enable routing in Linux systems.



BHARATIYA VIDYA BHAVAN'S

SARDAR PATEL INSTITUTE OF TECHNOLOGY

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai – 400058-India

Department of Computer Engineering